

Product Requirements Document (PRD)



Project : Next Gen CRM System

Prepared by:

Syed Tahmid Manzoor

North South University

Date: 27 October 2025

1. Executive Summary

1.1 Product Overview

A modern, user-friendly Customer Relationship Management (CRM) system designed for small and medium-sized businesses. The system features a sleek dark-themed interface with purple undertones, offering comprehensive customer management, communication tracking, and intelligent automation capabilities.

1.2 Target Users

- **Primary:** Small and Medium Business (SMB) owners
- **Secondary:** Sales teams, customer support teams, and administrative staff within SMBs

1.3 Value Proposition

A unified platform that combines customer relationship management, multi-channel communication tracking, intelligent automation, and conversational interfaces to streamline business operations and enhance customer engagement.

2. Product Objectives

2.1 Primary Goals

1. Provide an intuitive, modern interface for managing customer relationships
2. Centralize all customer interactions (emails, calls, complaints, logs) in one place
3. Enable both internal teams and external customers to interact through conversational interfaces
4. Automate routine tasks (email sorting, notifications, data entry)
5. Support multi-tenant architecture for scalability

2.2 Success Metrics

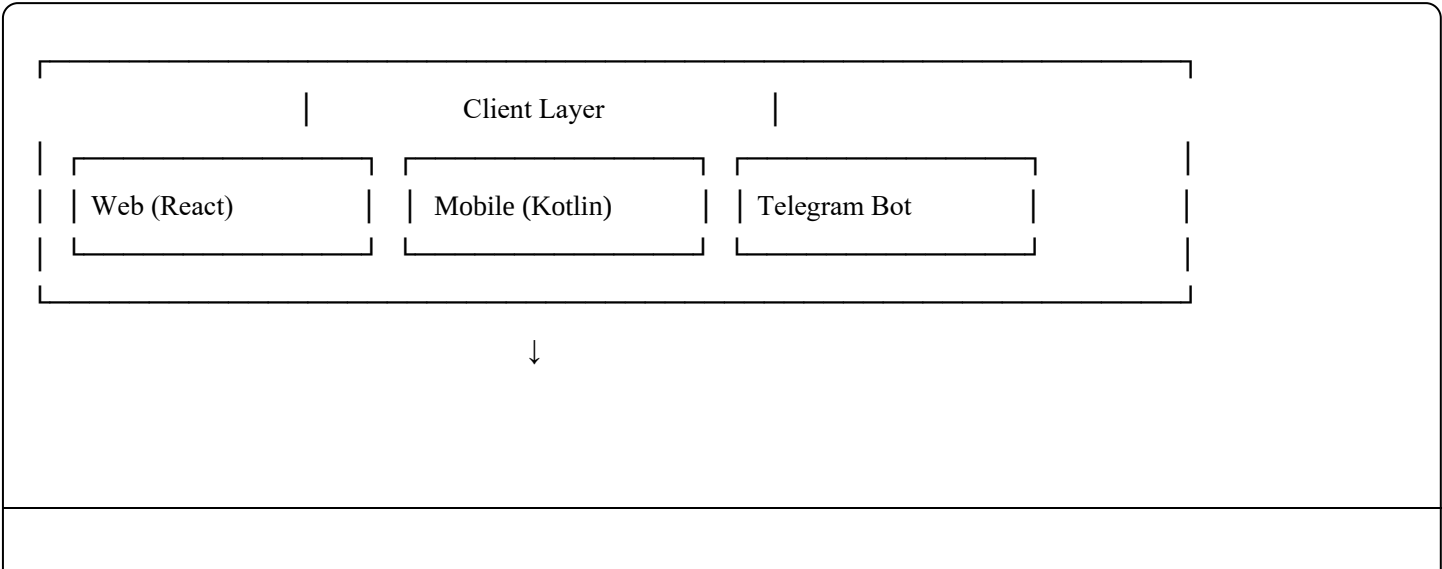
- User adoption rate within organizations
- Time saved on routine tasks (email sorting, data entry)
- Customer complaint resolution time
- User satisfaction score (NPS)
- System uptime and performance

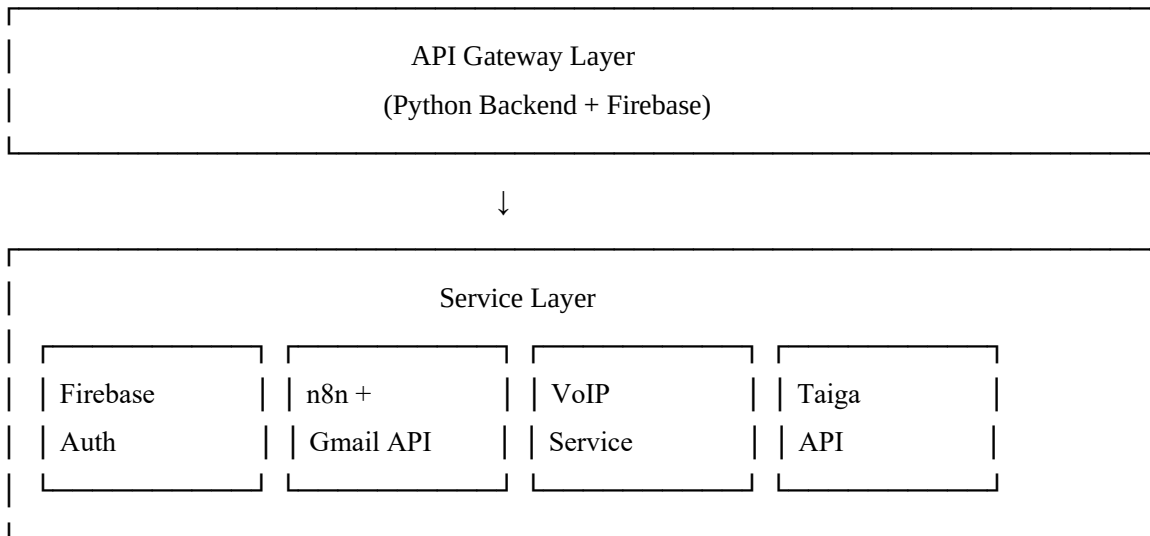
3. Technical Architecture

3.1 Tech Stack

Component	Technology	Purpose
Frontend	React + TailwindCSS	Web application UI
Mobile	Kotlin	Native Android application
Backend	Python (FastAPI)	API server and business logic
Database	Firebase Firestore	NoSQL database for real-time data
Authentication	Firebase Auth	User authentication and authorization
Email Integration	Gmail API via n8n	Email tracking and automation
Messaging	Telegram (BotFather)	Bot-based notifications and interactions
Issue Tracking	Taiga API	Third-party issue management integration
Phone System	VoIP Integration	Call functionality from app/web

3.2 System Architecture





4. API Specification

4.1 API Architecture

- **Protocol:** REST
- **Format:** JSON
- **Base URL:** `https://api.yourcrm.com/v1`
- **Authentication:** Bearer JWT tokens from Firebase Auth
- **Rate Limiting:** 100 requests/minute per user

4.2 Authentication Endpoints

POST /auth/login Login user

and get JWT token.

Request:

```
json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Response (200):

Json
{

```
"success": true,  
"data": {  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "refreshToken": "refresh_token_string",  
  "user": {  
    "id": "user_123",  
    "email": "user@example.com",  
    "displayName": "John Doe",  
    "role": "manager",  
    "tenantId": "tenant_abc"  
  }  
}  
}
```

POST **/auth/refresh**

Refresh expired JWT token.

Request:

```
json  
  
{  
  "refreshToken": "refresh_token_string"  
}
```

Response (200):

```
{{
```

```
json
{
  "success": true,
  "data": {
    "token": "new_jwt_token",
    "refreshToken": "new_refresh_token"
  }
}
```

POST /auth/logout

Invalidate current session.

Headers:

```
Authorization: Bearer {token}
```

Response (200):

```
json
{
  "success": true,
  "message": "Logged out successfully"
}
```

4.3 Customer Endpoints

GET /customers

Retrieve list of customers with pagination and filters.

Headers:

```
json
```

Authorization: Bearer {token}

X-Tenant-ID: {tenantId}

Query Parameters:

- page (integer, default: 1)
- limit (integer, default: 20, max: 100)
- search (string): Search in name,
email,company
- status (enum): active, inactive, prospect
- assignedTo (string): User ID
- tags (string): Comma-separated tags
- sortBy (string): name, createdAt, updatedAt
- sortOrder (enum): asc, desc

Response (200):

{{

```
"success": true,
"data": {
  "customers": [
    {
      "id": "cust_123",
      "companyName": "Acme Corp",
      "contactPerson": {
        "firstName": "John",
        "lastName": "Smith",
        "email": "john@acme.com",
        "phone": "+1234567890"
      },
      "status": "active",
      "tags": ["enterprise", "priority"],
      "assignedTo": "user_456",
      "createdAt": "2025-01-15T10:30:00Z",
      "updatedAt": "2025-01-20T14:22:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}
```

GET /customers/:id

Retrieve single customer details.

Response (200):

Json

{

{{

```
"success": true,
"data": {
  "id": "cust_123",
  "tenantId": "tenant_abc",
  "companyName": "Acme Corp",
  "contactPerson": {
    "firstName": "John",
    "lastName": "Smith",
    "email": "john@acme.com",
    "phone": "+1234567890",
    "role": "CEO"
  },
  "additionalContacts": [],
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zip": "10001",
    "country": "USA"
  },
  "tags": ["enterprise", "priority"],
  "customFields": {},
  "status": "active",
  "source": "referral",
  "assignedTo": "user_456",
  "createdAt": "2025-01-15T10:30:00Z",
  "updatedAt": "2025-01-20T14:22:00Z",
  "createdBy": "user_789"
}
```

POST /customers

Create new customer.

```
{{
```

Request:

Json

```
{  
  
  "companyName": "New Corp",  
  "contactPerson": {  
    "firstName": "Jane",  
    "lastName": "Doe",  
    "email": "jane@newcorp.com",  
    "phone": "+1987654321",  
    "role": "Manager"  
  },  
  "address": {  
    "street": "456 Oak Ave",  
    "city": "Boston",  
    "state": "MA",  
    "zip": "02101",  
    "country": "USA"  
  },  
  "tags": ["prospect"],  
  "status": "prospect",  
  "assignedTo": "user_456"  
}
```

Response (201):

json

```
{  
  "success": true,  
  "data": {  
    "id": "cust_456",  
    "message": "Customer created successfully"  
  }  
}
```

PUT /customers/:id

```
{{
```

Update existing customer.

```
"status": "active",  
"tags": ["enterprise", "priority", "vip"]  
}
```

Request: (partial update supported)

json

Response (200):

```
json  
  
{  
  "success": true,  
  "data": {  
    "id": "cust_123",  
    "message": "Customer updated successfully"  
  }  
}
```

DELETE /customers/:id Soft delete

customer (mark as inactive).

Response (200):

```
json  
  
{  
  "success": true,  
  "message": "Customer deleted successfully"  
}
```


{{

4.4 Log Entry Endpoints GET

/customers/:customerId/logs

Retrieve logs for specific customer.

Query Parameters:

- **type** (enum): call, email, meeting, note, sample, other
- **threadId** (string): Filter by conversation thread
- **startDate** (ISO date)

- `endDate` (ISO date)
- `page`, `limit`

Response (200):

```
json
{
  "success": true,
  "data": {
    "logs": [
      {
        "id": "log_789",
        "customerId": "cust_123",
        "threadId": "thread_001",
        "type": "call",
        "subject": "Follow-up call",
        "description": "Discussed project timeline and budget",
        "attachments": [],
        "tags": ["follow-up"],
        "createdAt": "2025-01-22T09:15:00Z",
        "createdBy": "user_456"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 45
    }
  }
}
```

POST `/customers/:customerId/logs`

Create new log entry.

Request:

json

```
{
  "type": "meeting",
  "threadId": "thread_001",
  "subject": "Quarterly Review",
  "description": "Reviewed Q4 performance metrics",
  "tags": ["review", "quarterly"],
  "attachments": [
    {
      "filename": "Q4_report.pdf",
      "url": "https://storage.example.com/files/report.pdf",
      "size": 1024000,
      "type": "application/pdf"
    }
  ]
}
```

Response (201):

json

```
{
  "success": true,
  "data": {
    "id": "log_890",
    "message": "Log created successfully"
  }
}
```

PUT /logs/:id

Update log entry.

DELETE **/logs/:id**

Delete log entry.

4.5 Complaint Endpoints

GET /complaints

List all complaints with filters.

Query Parameters:

- `status` (enum): new, acknowledged, in_progress, resolved, closed
- `severity` (enum): low, medium, high, critical
- `category` (string)
- `assignedTo` (string)
- `customerId` (string)
- `page, limit`

Response(200):

json

```
{
  "success": true,
  "data": {
    "complaints": [
      {
        "id": "comp_123",
        "ticketNumber": "COMP-2025-001",
        "customerId": "cust_123",
        "title": "Product defect",
        "category": "product",
        "severity": "high",
        "status": "in_progress",
        "assignedTo": "user_456",
        "createdAt": "2025-01-20T11:00:00Z",
        "sla": {
          "responseDeadline": "2025-01-20T15:00:00Z",
          "resolutionDeadline": "2025-01-22T11:00:00Z"
        }
      }
    ],
    "pagination": {}
  }
}
```

GET /complaints/:id

Get complaint details including timeline.

Response (200):

json

```
{
  "success": true,
  "data": {
    "id": "comp_123",
    "ticketNumber": "COMP-2025-001",
    "tenantId": "tenant_abc",
    "customerId": "cust_123",
    "title": "Product defect",
    "description": "Product stopped working after 2 weeks",
    "category": "product",
    "severity": "high",
    "status": "in_progress",
    "assignedTo": "user_456",
    "taigaIssueId": "taiga_789",
    "timeline": [
      {
        "timestamp": "2025-01-20T11:00:00Z",
        "action": "created",
        "userId": "user_123",
        "details": "Complaint submitted"
      },
      {
        "timestamp": "2025-01-20T11:30:00Z",
        "action": "status_changed",
        "userId": "user_456",
        "details": "Status changed to acknowledged"
      }
    ],
    "internalComments": [],
    "customerUpdates": [],
    "attachments": [],
    "createdAt": "2025-01-20T11:00:00Z",
    "createdBy": "user_123"
  }
}
```

POST /complaints

Create new complaint.

Request:

```
json

{
  "customerId": "cust_123",
  "title": "Billing issue",
  "description": "Charged twice for same invoice",
  "category": "billing",
  "severity": "medium",
  "attachments": []
}
```

Response (201):

```
json

{
  "success": true,
  "data": {
    "id": "comp_456",
    "ticketNumber": "COMP-2025-002",
    "message": "Complaint created successfully"
  }
}
```

PUT /complaints/:id/status

Update complaint status.

Request:

json

```
{
  "status": "resolved",
  "resolutionNotes": "Refund issued",
  "customerSatisfaction": 4
}
```

POST /complaints/:id/comments

Add internal comment.

Request:

json

```
{
  "comment": "Contacted customer, awaiting response"
}
```

POST /complaints/:id/updates

Send update to customer.

Request:

json

```
{
  "message": "We're working on your issue and will update you by end of day"
}
```

4.6 Search Endpoint

GET /search

Global search across all entities.

Query Parameters:

- `q` (string, required): Search query
- `type` (enum): customers, logs, complaints, all (default)
- `limit` (integer)

Response (200):

json

```
{
  "success": true,
  "data": {
    "results": [
      {
        "type": "customer",
        "id": "cust_123",
        "title": "Acme Corp",
        "subtitle": "john@acme.com",
        "highlights": ["Acme Corp matches your search"],
        "url": "/customers/cust_123"
      },
      {
        "type": "log",
        "id": "log_456",
        "title": "Meeting with Acme Corp",
        "subtitle": "2025-01-20",
        "highlights": ["...discussed Acme Corp project..."],
        "url": "/logs/log_456"
      }
    ],
    "total": 12
  }
}
```

4.7 User Management Endpoints

GET /users

List users in tenant.

POST /users/invite

Invite new user to tenant.

Request:

```
json

{
  "email": "newuser@example.com",
  "displayName": "New User",
  "role": "sales_rep"
}
```

PUT /users/:id/role

Change user role.

DELETE /users/:id

Deactivate user.

4.8 Email Integration Endpoints

GET /customers/:customerId/emails

Get email history for customer.

POST /emails/send

Send email from CRM.

Request:

json

```
{
  "to": "customer@example.com",
  "subject": "Follow-up",
  "body": "Thank you for meeting with us...",
  "customerId": "cust_123",
  "templateId": "template_001"
}
```

4.9 Call Endpoints

POST /calls/initiate

Start a call from CRM.

Request:

json

```
{
  "customerId": "cust_123",
  "phoneNumber": "+1234567890"
}
```

Response (200):

json

```
{
  "success": true,
  "data": {
    "callId": "call_789",
    "status": "initiated",
    "sipUrl": "sip:call_789@voip.example.com"
  }
}
```

GET **/customers/:customerId/calls**

Get call history for customer.

4.10 Analytics Endpoints

GET **/analytics/dashboard**

Get dashboard metrics.

Response (200):

```
json

{
  "success": true,
  "data": {
    "totalCustomers": 150,
    "activeComplaints": 8,
    "complaintsThisWeek": 12,
    "avgResolutionTime": "2.5 days",
    "logsThisMonth": 234,
    "recentActivity": []
  }
}
```

GET **/analytics/complaints**

Get complaint analytics.

4.11 Webhook Endpoints

POST **/webhooks/gmail**

Receive Gmail webhooks from n8n.

POST **/webhooks/taiga**

Receive Taiga issue updates.

POST /webhooks/telegram

Receive Telegram bot messages.

4.12 Error Responses

All endpoints follow consistent error format:

Response (4xx/5xx):

```
json

{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid email format",
    "details": {
      "field": "email",
      "value": "invalid-email"
    }
  }
}
```

Common Error Codes:

- `UNAUTHORIZED` (401)
 - `FORBIDDEN` (403)
 - `NOT_FOUND` (404)
 - `VALIDATION_ERROR` (400)
 - `RATE_LIMIT_EXCEEDED` (429)
 - `INTERNAL_SERVER_ERROR` (500)
-

5. Wireframes & UI Mockups

5.1 Design System

Color Palette

Primary Background: #1a1a2e
Secondary Background: #16213e
Card Background: #0f3460
Primary Accent: #6C63FF (Purple)
Secondary Accent: #9F7AEA (Light Purple)
Success: #10B981
Warning: #F59E0B
Error: #EF4444
Text Primary: #F9FAFB
Text Secondary: #D1D5DB
Border: #374151

Typography

Headings: Inter (Bold)
H1: 32px / 2rem
H2: 24px / 1.5rem
H3: 20px / 1.25rem
H4: 18px / 1.125rem

Body: Inter (Regular)
Large: 16px / 1rem
Medium: 14px / 0.875rem
Small: 12px / 0.75rem

Spacing Scale

xs: 4px
sm: 8px
md: 16px
lg: 24px
xl: 32px
2xl: 48px
3xl: 64px

5.2 Dashboard (Home Screen)

CRM Logo

[Search...]

Settings

150

Customers

12

Unread Emails

8

Open Complaints

Recent Activity

[View All =>]

John called Acme Corp

2 hours ago

New email from TechStart

4 hours ago

Complaint #123 resolved

Yesterday

Sarah added log for XYZ Inc

Yesterday

Quick Actions

[+ Add Customer]

[Create Log]

[Report Issue]

Upcoming Tasks

[View All =>]

- ☐ Follow up with Acme Corp Due: Today
- ☐ Send proposal to NewCo Due: Tomorrow

Key Features:

- Metric cards with live data
- Activity feed with icons and timestamps
- Quick action buttons (purple accent)
- Task list with checkboxes
- Global search in header
- Notifications bell icon
- User profile dropdown

5.3 Customer List View

Customers				[+ Add]			
[Search customers...] [▼ Filter] [⚙ Sort] [📊 Grid]							
Company	Contact	Status	Actions				
🏢 Acme Corp Enterprise Assigned to: Sarah J.	john@acme.com	🟡 Active 👁 ✎ 🗑					
🏢 TechStart	jane@tech.com	🟡 Prospect 👁 ✎ 🗑					

SaaS	
Assigned to: Mike K.	
 NewCo Ltd  bob@newco.com  Active   	
Manufacturing	
Assigned to: Sarah J.	
Showing 1-20 of 150	[<= Prev] Page 1 of 8 [Next =>]

Key Features:

- Search bar with real-time filtering
- Filter dropdown (by status, tags, assigned user)
- Sort options
- Grid/List view toggle
- Inline actions (view, edit, delete)
- Visual status indicators (colored dots)
- Tags displayed under company name
- Pagination controls

5.4 Customer Detail View

Back to Customers	
 Acme Corp john@acme.com +1 234 567 8900	Quick Actions [ Call] [ Email] [ Add Log]

 Active

 Enterprise, Priority

 Assigned: Sarah J.

[Timeline]

[Logs]

[Complaints]

[Files]

[Details]

Timeline

[Filter]

Jan 22, 2025 09:15 AM

 Sarah called John Smith

Discussed Q1 project timeline and budget

[View Details]

Jan 20, 2025 02:30 PM

 Email sent: Follow-up from meeting

Sent quote for new project

[View Email]

Jan 18, 2025 11:00 AM

 Meeting: Quarterly Review

Reviewed Q4 performance metrics

 Q4_report.pdf

[Load More]

Key Features:

- Customer info card (left sidebar)
- Quick actions panel
- Tabbed navigation (Timeline, Logs, Complaints, Files)
- Visual timeline with icons for different event types
- Expandable event details
- File attachments shown inline
- Filter option for timeline
- Infinite scroll / Load more

5.5 Complaint Management (Kanban View)

Complaints

[List]

[Board]

[+ New]

</

002	004	009	
Service	Product	Billing	
issue	query	solved	

Key Features:

- Kanban board with drag-and-drop
- Color-coded severity indicators (High, Medium, Low)
- Card shows: ticket number, category, customer, assigned user
- Count of complaints in each column
- Toggle between Board and List view
- Search and filter capabilities
- Click card to open detail modal

5.6 Complaint Detail Modal

Complaint #COMP-2025-001		[X]
Status: In Progress		Severity: High
Category: Product		Assigned: Sarah J.
Customer: Acme Corp		Created: Jan 20, 2025
Title:		
Product stopped working after 2 weeks		

Description:

The device we purchased started malfunctioning after just 2 weeks of use. It powers on but screen remains blank. Customer needs urgent replacement.

SLA Timeline:

Response due:  Met (responded in 30 min)

Resolution due:  1 day 4 hours remaining

Linked Taiga Issue: #789 [View in Taiga]

Timeline

Jan 20, 11:00 AM - Complaint created by John Smith
Jan 20, 11:30 AM - Status changed to Acknowledged
Jan 20, 02:15 PM - Assigned to Sarah J.
Jan 20, 02:45 PM - Status changed to In Progress
Jan 21, 09:30 AM - Sarah added internal comment

Internal Comments (Not visible to customer)

Sarah J. - Jan 21, 09:30 AM
Contacted supplier, replacement will ship today.
Will update customer once tracking info received.

[Add Internal Comment]

Customer Updates

Sarah J. - Jan 20, 03:00 PM
We're working on your issue. A replacement unit will be shipped within 24 hours.

[Send Update to Customer]

Actions:		
[Change Status]	[Reassign]	[Mark Resolved]

Key Features:

- Full complaint details with all metadata
- SLA tracking with visual indicators
- Linked Taiga issue with direct link
- Complete timeline of all actions
- Separate sections for internal comments vs customer updates
- Action buttons for status change, reassignment
- Clear visual hierarchy

5.7 Conversational Interface (Internal Chatbot)

AI Assistant

[-][X]

Hello Sarah! How can I help you today?

Show me all complaints from this week

I found 12 complaints from this week:

High Priority (3)

• COMP-001 - Product defect (Acme Corp)

• COMP-005 - Service issue (XYZ Inc)

• COMP-009 - Delivery delay (NewCo)

Medium Priority (5)

Low Priority (4)

Would you like me to:
[Show details] [Filter by status] [Export list]

Create a log for Acme Corp

I'll help you create a log for Acme Corp.

What type of log would you like to create?
[Call] [Email] [Meeting] [Note] [Sample]

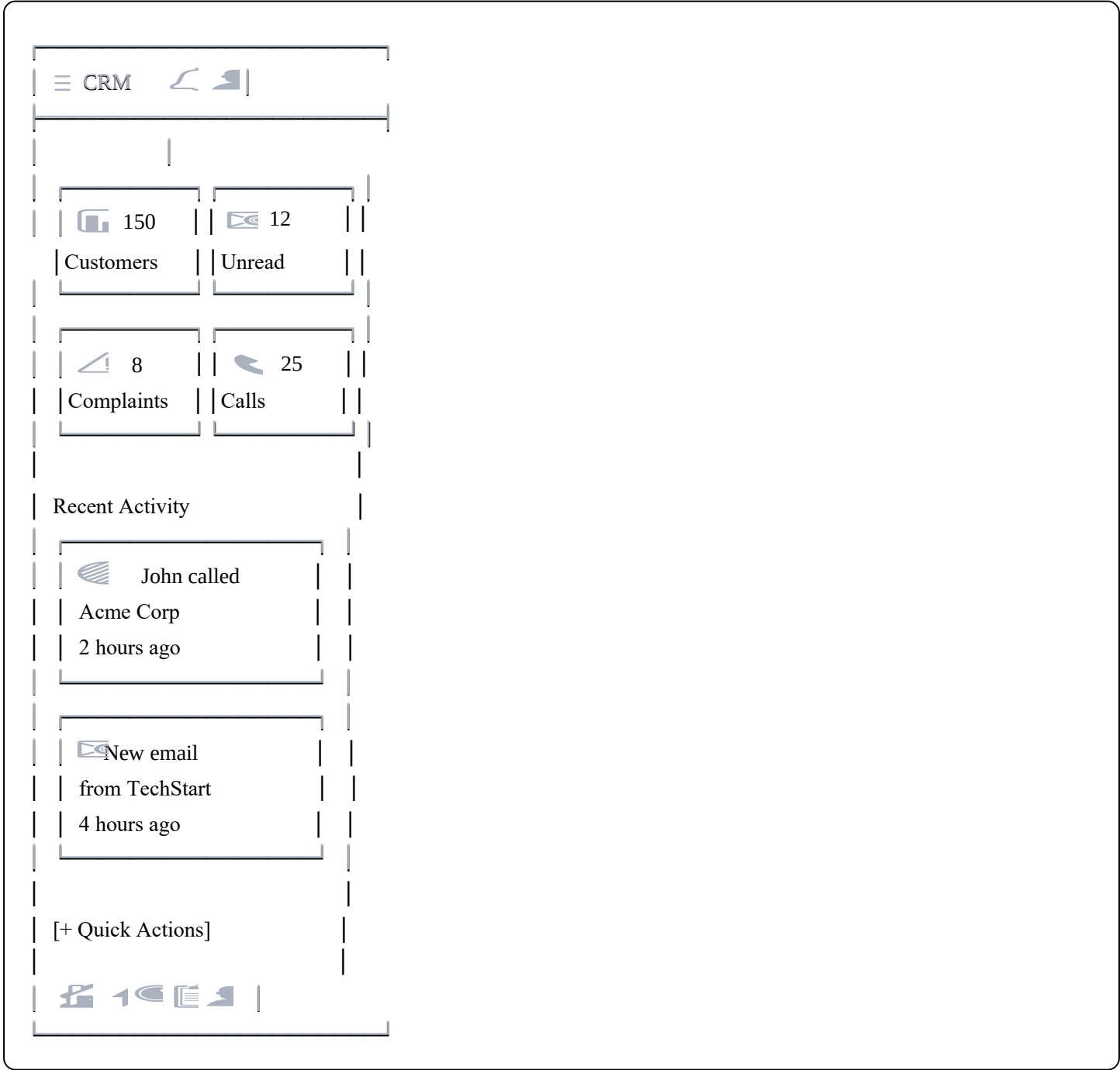
Type your message or use voice...

Key Features:

- Floating chat widget (can minimize)
- Natural language queries
- Structured responses with formatting
- Quick action buttons in responses
- Voice input icon (🎤)
- File attachment option (📎)
- Context-aware suggestions
-

5.8 Mobile App (Android - Kotlin)

Home Screen



Customer Detail (Mobile)

<= Acme Corp |



Acme Corp

john@acme.com

+1 234 567 8900



Active



Call]



Email]

Timeline Logs 



Today, 09:15 AM

Called John Smith

Discussed Q1...

[View]



Yesterday

Email sent

Follow-up from...

[View]



Jan 18

Quarterly Review

Reviewed Q4...

[View]

[+ Add Log]

Mobile-Specific Features:

- Bottom navigation bar
- Swipe gestures (swipe to archive, delete)
- Native dialer integration
- Camera integration for attachments
- Voice input always available
- Offline mode with sync
- Push notifications
- Biometric authentication

5.9 Email Integration View

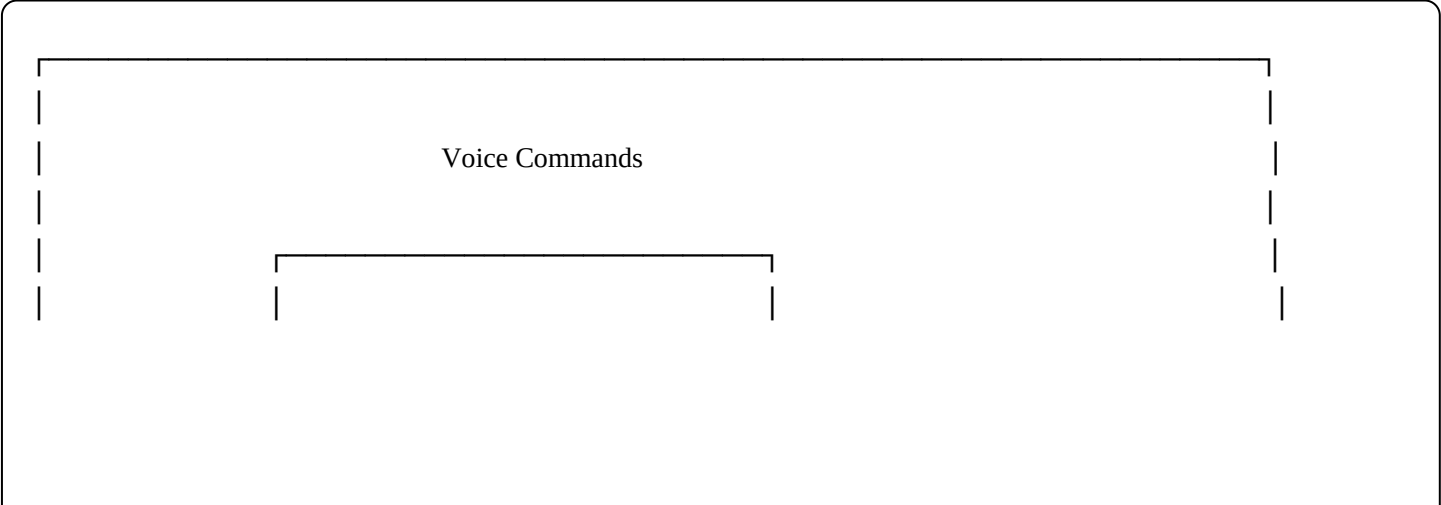
Emails		[Compose]	[Sync]
[Inbox] [Sent] [Auto-sorted]			
Auto-sorted Emails		[Settings]	
 Sales Inquiry Unread			
From: prospect@newcompany.com			
Re: Product pricing			
Auto-linked to:  NewCompany (Prospect)			
Category:  Sales Priority:  Medium			
Assigned to: Sarah J.			
2 hours ago			
 Complaint Unread			
From: john@acme.com			
Re: Product not working			
Auto-linked to:  Acme Corp			
Category:  Complaint Priority:  High			

Assigned to: Support Team		
Auto-created complaint: #COMP-001		
4 hours ago		
✉ Follow-up	Read	
From: jane@techstart.com		
Re: Meeting summary		
Auto-linked to: ➡ TechStart		
Category: 🔗 Follow-up Priority: 🟡 Low		
Yesterday		

Key Features:

- Tabbed view (Inbox, Sent, Auto-sorted)
- Visual indicators for auto-categorization
- Auto-linking to customer records shown
- Priority and category badges
- Auto-assigned user displayed
- Quick sync button
- Settings for automation rules
- Compose button for sending from CRM

5.10 Voice Command Interface



Listening...

"Show me all customers
from last month"

[Cancel]

Try saying:

- "Create a log for [customer name]"
- "Show complaints assigned to me"
- "Call [customer name]"
- "Search for [query]"
- "Go to dashboard"

Processing State:

 Processing...

"Show me all customers
from last month"

Confirmation State:

✓ Command Recognized

	Showing 23 customers	
	from last month	
[View Results]		

5.11 Settings & Configuration

Settings			
[Account] [Organization] [Users] [Integrations] [Billing]			
Organization Settings			
Organization Name			
Acme Corporation			
Subdomain			
acme .yourcrm.com			
Branding			
Logo: [Upload] [Current: acme-logo.png]			
Primary Color: #6C63FF			
Secondary Color: #9F7AEA			
[Save Changes]			
User Management			
NameEmailRoleActions			

	Sarah Johnson	sarah@acme.com	Admin				
	Mike Anderson	mike@acme.com	Manager				
	John Smith	john@acme.com	Sales				
[+ Invite User]							
Integrations							
		Gmail	✓ Connected	[Configure]	[Test]		
		Telegram	✓ Connected	[Configure]	[Test]		
		Taiga	✓ Connected	[Configure]	[Test]		
		VoIP	 Not setup	[Connect]			

6. Feature Requirements

6.1 Core Features (MVP - Phase 1)

Feature 1: Comprehensive Logging System

Priority: P0 (Critical)

Description: Record and track all interactions, samples, and people associated with customers.

Requirements:

- **Log Types:**
 - Activity logs (calls, emails, meetings, notes)
 - Sample logs (product samples sent, demos conducted)
 - People logs (contacts, stakeholders, decision-makers)

- **Log Attributes:**
 - Timestamp (auto-generated)
 - User who created the log
 - Associated customer/company
 - Log type and category
 - Description/notes
 - Attachments (files, images)
 - Tags for categorization
- **Functionality:**
 - Create, read, update, delete (CRUD) operations
 - Bulk import/export capabilities
 - Auto-logging from email and call integrations
 - Rich text editor for detailed notes
 - File attachments (max 10MB per file)

Acceptance Criteria:

- Users can create different types of logs in < 30 seconds
 - All logs are timestamped and attributed to the creator
 - Logs are immediately synced across all platforms
 - Support for 100-300 daily log entries without performance degradation
-

Feature 2: Customer History & Timeline

Priority: P0 (Critical)

Description: Maintain comprehensive customer history with support for multiple simultaneous interactions with the same customer.

Requirements:

- **Timeline View:**
 - Chronological display of all customer interactions
 - Visual timeline with color-coded events
 - Filtering by date range, interaction type, team member
 - Expandable/collapsible detail views
- **Multiple Simultaneous Histories:**
 - Support for concurrent projects/deals with same customer
 - Separate conversation threads
 - Ability to link related threads
 - Context switching between different interaction streams
- **History Types:**
 - Email conversations
 - Call logs
 - Meeting notes
 - Complaints
 - Samples sent
 - Quotes/proposals
 - Invoices
 - Support tickets

Data Model:

```

Customer
├── Basic Info (name, company, contact)
├── Interaction Threads []
│   ├── Thread 1 (Project A)
│   │   └── Events []
│   └── Thread 2 (Support Issue B)
│       └── Events []
└── Metadata (tags, custom fields)
  
```


Acceptance Criteria:

- Display complete customer history in < 2 seconds
 - Support minimum 50 events per customer without lag
 - Clear visual separation of concurrent interaction threads
 - Quick switch between different threads
-

Feature 3: Advanced Search

Priority: P0 (Critical)

Description: Powerful search functionality across all CRM data.

Requirements:

- **Search Scope:**
 - Customers (name, company, email, phone)
 - Logs and notes
 - Email content
 - Complaints
 - Tags and custom fields
- **Search Features:**
 - Full-text search
 - Fuzzy matching for typos
 - Filters (date range, type, status, user)
 - Advanced boolean operators (AND, OR, NOT)
 - Saved searches
 - Search suggestions/autocomplete
 - Recent searches history

- **Performance:**
 - Results displayed in < 1 second
 - Support for searching across 200-500 customer records
 - Real-time search as user types (debounced)

UI Components:

- Global search bar in header
- Advanced search modal with filters
- Search results with highlighting
- Quick actions on search results

Acceptance Criteria:

- Search returns relevant results in < 1 second
 - Fuzzy matching catches common typos
 - Users can filter results by multiple criteria
 - Search works across all platforms (web, mobile)
-

Feature 4: Email Integration

Priority: P0 (Critical)

Description: Integrate Gmail API via n8n for email tracking and management.

Requirements:

- **Email Sync:**
 - Two-way sync with Gmail
 - Auto-link emails to customer records
 - Thread/conversation grouping
 - Attachment handling

- **Email Features:**
 - Send emails from within CRM
 - Email templates
 - Email scheduling
 - Read receipts (if available)
 - BCC to CRM for auto-logging
- **Automation** (See Feature 8):
 - Auto-categorization of emails
 - Auto-assignment to team members
 - Priority flagging
- **Email Display:**
 - Threaded conversation view
 - Rich HTML rendering
 - Attachment preview
 - Reply/forward/compose

Integration via n8n:

- Webhook triggers for new emails
- Workflow automation for email routing
- Custom email processing rules

Acceptance Criteria:

- Emails sync within 5 minutes of receipt
 - Emails automatically linked to correct customer (90% accuracy)
 - Users can send emails without leaving CRM
 - Email threads properly grouped
-

Feature 5: Complaint Management System

Priority: P1 (High)

Description: Dedicated module for logging, tracking, and resolving customer complaints efficiently.

Requirements:

- **Complaint Lifecycle:**
 - States: New → Acknowledged → In Progress → Resolved → Closed
 - Status tracking with timestamps
 - Resolution time tracking
 - Escalation mechanism
- **Complaint Attributes:**
 - Customer information
 - Complaint category (product, service, billing, etc.)
 - Severity level (Low, Medium, High, Critical)
 - Description and details
 - Assigned team member
 - Resolution notes
 - Customer satisfaction rating (post-resolution)
- **Features:**
 - SLA tracking (response time, resolution time)
 - Auto-assignment rules
 - Email notifications at each status change
 - Internal comments (not visible to customer)
 - Customer-facing updates
 - Attachment support (screenshots, documents)

- **Analytics:**
 - Complaint trends by category
 - Average resolution time
 - Team member performance
 - Complaint volume over time

Acceptance Criteria:

- Complaints can be created in < 1 minute
 - Status updates trigger notifications
 - Overdue complaints highlighted
 - Resolution time calculated automatically
-

Feature 6: Third-Party Issue Tracking (Taiga Integration)

Priority: P1 (High)

Description: Integration with Taiga open-source project management platform for advanced issue tracking.

Requirements:

- **Taiga Connection:**
 - OAuth authentication with Taiga
 - API integration for bidirectional sync
 - Project mapping (CRM customer → Taiga project)
- **Functionality:**
 - Create Taiga issues from CRM
 - Sync issue status from Taiga to CRM
 - Display linked Taiga issues in customer view
 - Comment synchronization
 - Attachment sharing

- **Use Cases:**
 - Technical issues requiring development team
 - Complex customer requests needing project management
 - Feature requests tracking
 - Bug reports from customers

Data Flow:

CRM Complaint/Issue → Taiga Issue
Taiga Status Update → CRM Update → Customer Notification

Acceptance Criteria:

- Issues created in Taiga within 30 seconds
 - Status syncs every 5 minutes
 - Clear visual indication of linked Taiga issues
 - One-click navigation to Taiga issue
-

Feature 7: Call Integration (App/Web)

Priority: P1 (High)

Description: Enable users to make calls directly from the CRM by integrating with existing phone systems.

Requirements:

- **Integration Options:**
 - SIP/VoIP integration (e.g., Twilio, Vonage)
 - Click-to-call from customer profiles
 - Support for existing PBX systems
- **Call Features:**
 - Initiate calls from web and mobile app
 - Call logging (auto-create log entry)
 - Call recording (with consent notification)
 - Call notes during/after call
 - Call duration tracking
 - Caller ID display
- **Call Log Attributes:**
 - Customer name/number
 - Date and time
 - Duration
 - Direction (inbound/outbound)
 - Team member who handled call
 - Call outcome/disposition
 - Notes/summary
 - Recording link (if recorded)
-

-
- **Phone System Integration:**
 - WebRTC for browser-based calls
 - Native dialer integration for mobile

Call transfer capabilities

Hold/mute functionality

Acceptance Criteria:

- Click-to-call initiates call in < 3 seconds
 - All calls automatically logged
 - Call recordings accessible from customer history
 - Works on both web and mobile platforms
-

Feature 8: Automated Email Sorting

Priority: P1 (High)

Description: Intelligent automation to categorize, route, and prioritize incoming emails.

Requirements:

- **Auto-Categorization:**
 - Machine learning or rule-based classification
 - Categories: Sales inquiry, Support request, Complaint, Follow-up, General
 - Confidence score for each classification
 - Manual override option
- **Auto-Assignment:**
 - Route emails to appropriate team members
 - Round-robin or skill-based routing
 - Load balancing across team
 - Escalation rules for high-priority emails
-

-
- **Priority Detection:**
 - Identify urgent emails (keywords, sentiment analysis)
 - Flag VIP customers
 - Deadline detection (e.g., "by end of day")
- **Automation Rules:**
 - User-defined rules (if subject contains X, assign to Y)
 - Templates for common responses
 - Auto-reply for specific scenarios
 - Spam filtering

Implementation via n8n:

- Email received → n8n workflow triggered
- Content analysis (keywords, sender, subject)
- Apply classification rules
- Create CRM record and assign
- Notify assigned team member

Acceptance Criteria:

- 80% accuracy in email categorization
- Emails auto-assigned within 2 minutes
- High-priority emails flagged immediately
- Users can customize automation rules

Feature 9: Conversational Interface (Dual Purpose)

Priority: P1 (High)

Description: Two distinct conversational interfaces:

1. **Internal:** Chatbot for CRM users to interact with the system using natural language
 -

-

2. **External:** Conversational interface for end-customers to interact with the business **Requirements:**

9A. Internal Conversational Interface

- **Capabilities:**
 - Natural language queries: "Show me all customers from last month"
 - Quick actions: "Create a log for Acme Corp"
- Data retrieval: "What's the status of complaint #123?"

-

-

Search: "Find customers in New York"

- Analytics: "How many complaints this week?"

- **Integration Points:**

- Available in web app, mobile app, and Telegram bot
- Context-aware (knows current page/customer)
- Access to full CRM database (with permissions)

- **Technology:**

- Natural Language Processing (NLP) engine
- Intent recognition
- Entity extraction
- Conversation memory/context

9B. External Conversational Interface

- **Capabilities:**

- Customer can check order status
- Submit support requests
- Schedule appointments
- Get business hours/information
- FAQ responses

- **Channels:**

- Website chat widget
- Telegram bot for customers
- Email responses (limited)

- **Features:**
 - Auto-create customer records for new interactions
 - Escalate to human when needed
 - Handoff with conversation history
 - Multi-language support (future)

Shared Features:

- Conversation history
- Quick reply buttons
- Rich media support (images, files)
- Typing indicators
- Read receipts

Acceptance Criteria:

- Internal bot correctly interprets 70% of queries
 - External bot handles 50% of customer inquiries without escalation
 - Conversation handoff to human is seamless
 - Response time < 2 seconds
-

Feature 10: Voice Commands

Priority: P2 (Medium)

Description: Enable users to control the CRM using natural speech rather than typing commands.

Requirements:

- **Voice Input:**
 - Speech-to-text conversion
 - Support for common accents/dialects
 - Noise cancellation
 -

Push-to-talk and always-listening modes

- **Voice Commands:**
 - Navigation: "Go to customer list"
 - Search: "Find John Smith"
 - Create: "Create a new log for Acme Corp"
 - Update: "Update complaint status to resolved"
 - Query: "Show me today's tasks"
- **Voice Feedback:**
 - Text-to-speech for responses (optional)
 - Visual confirmation of recognized command
 - Error correction ("Did you mean...")
- **Platform Support:**
 - Web (browser speech API)
 - Mobile (native speech recognition)
 - Hands-free operation mode

Privacy & Security:

- Voice data not stored permanently
- User consent required
- Secure transmission of voice data

Acceptance Criteria:

- 85% accuracy in command recognition
 - Commands executed within 3 seconds
 - Clear visual/audio feedback for each command
 - Works in moderately noisy environments
-

Feature 11: Multiple Clients Access Management

Priority: P0 (Critical)

Description: Comprehensive access control supporting both role-based permissions within organizations and multi-tenant architecture for independent companies.

Requirements:

11A. Multi-Tenancy

- **Tenant Isolation:**
 - Complete data separation between companies
 - Unique subdomain or tenant identifier (e.g., acme.yourcrm.com)
 - Separate databases or partitioned data
 - No cross-tenant data leakage
- **Tenant Management:**
 - Self-service tenant creation
 - Tenant admin dashboard
 - Billing per tenant (if applicable)
 - Custom branding per tenant (logo, colors)
 - Usage analytics per tenant

11B. Role-Based Access Control (RBAC)

- **Predefined Roles:**
 - **Super Admin:** Full system access (your role)
 - **Tenant Admin:** Full access within their organization
 - **Manager:** View all data, manage team members, create/edit records
 - **Sales Rep:** View and edit own customers, limited access to others
 - **Support Agent:** View customers, manage complaints, limited edit access
 - **Viewer:** Read-only access

- **Permission Granularity:**
 - Module-level (access to Customers, Complaints, Logs, etc.)
 - Action-level (Create, Read, Update, Delete)
 - Data-level (own records vs. all records)
 - Field-level (sensitive fields hidden from certain roles)
- **Permission Matrix Example:**

Role	View All	Edit All	Delete	Manage Users	Access Settings
Tenant Admin	✓	✓	✓	✓	✓
Manager	✓	✓	✓	✗	✗
Sales Rep	Own only	Own only	✗	✗	✗
Support	✓	Limited	✗	✗	✗
Viewer	✓	✗	✗	✗	✗

11C. User Management

- **Features:**
 - Invite users via email
 - Assign roles on invitation
 - Change user roles
 - Deactivate/reactivate users
 - User activity audit log
 - Session management
- **Security:**
 - Firebase Auth for authentication
 - JWT tokens for API access
 - Password policies
 - Two-factor authentication (future)
 - IP whitelisting (optional)

Acceptance Criteria:

- Complete data isolation between tenants verified
 - Users cannot access data outside their permissions
 - Role changes take effect immediately
 - Support for 5-15 users per tenant
 - Audit trail for all permission changes
-

7. Data Models

7.1 Core Entities

Customer

javascript


```
{{
  idid: string string,

  tenantId: string,
  companyName: string,
  contactPerson: {
    firstName: string,
    lastName: string,
    email: string,
    phone: string,
    role: string
  },
  additionalContacts: [{
    firstName: string,
    lastName: string,
    email: string,
    phone: string,
    role: string
  }],
  address: {
    street: string,
    city: string,
    state: string,
    zip: string,
    country: string
  },
  tags: [string],
  customFields: object,
  status: enum['active', 'inactive', 'prospect'],
  source: string,
  assignedTo: string (userId),
  createdAt: timestamp,
  updatedAt: timestamp,
  createdBy: string (userId)
}
```

```
{  
  idid: string string,
```

Log Entry

```
javascript
```

```
tenantId: string,  
customerId: string,  
threadId: string (optional, for grouping related logs),  
type: enum['call', 'email', 'meeting', 'note', 'sample', 'other'],  
category: string,  
subject: string,  
description: string (rich text),  
attachments: [{  
  filename: string,  
  url: string,  
  size: number,  
  type: string  
}],  
tags: [string],  
createdAt: timestamp,  
createdBy: string (userId),  
updatedAt: timestamp  
}
```

Complaint

```
{  
  id: string,
```

javascript

```
{  
  id: string,
```

```
  ticketNumber: string,
```

```
{{
  idid: string string,

stringstring ((autoauto--
generatedgenerated)),
tenantIdtenantId: string string,
customerIdcustomerId: string string,
titletitle: string string,
descriptiondescription: string string,
  categorycategory: enumenum[['product'product',
'service'service', 'billing'billing', 'delivery'delivery',
'other'other']],,  severityseverity: enumenum[['low'low',
'medium'medium', 'high'high', 'critical'critical']],,
  statusstatus: enumenum[['new'new',
'acknowledged'acknowledged', 'in_progress'in_progress',
'resolved'resolved', 'closed'closed']],,  assignedToassignedTo:
stringstring ((userIduserId)),  taigaIssueIdtaigaIssueId:
stringstring ((optionaloptional)),  prioritypriority: number
number,,  sla: {{
  responseDeadlineresponseDeadline:
timestamp timestamp,,
resolutionDeadlineresolutionDeadline: timestamp
timestamp
  }},,
timeline
line: [[{{

timestamptimestamp:
timestamp timestamp,,
actionaction: string
string,,
userIduserId: string
string,,
detailsdetails: string
string
  }]]],  resolutionresolution: {{
notesnotes: string string,,
```

```
{  
  id: string!,  
  
  resolvedAt: timestamp!  
  resolvedBy: string! (userId!),  
  customerSatisfaction: number! (1-5),  
}
```

```
internalComments: [{  
  user: string!,  
  comment: string!,  
  timestamp: timestamp!  
}]
```

```
customerUpdates: [{  
  message: string!,  
  timestamp: timestamp!,  
  sentBy: string! (userId!),  
  attachments: [object!],  
  createdAt: timestamp!  
}]
```

```
    createdBy: string (userId)
  }
```

User

javascript

```
{
  id: string,
  tenantId: string,
  email: string,
  displayName: string,
  role: enum['super_admin', 'tenant_admin', 'manager', 'sales_rep', 'support_agent', 'viewer'],
  permissions: [string],
  phone: string,
  avatar: string (url),
  status: enum['active', 'inactive'],
  lastLogin: timestamp,
  createdAt: timestamp,
  preferences: {
    theme: string,
    notifications: object,
    language: string
  }
}
```

Tenant

javascript

```
{
  id: string,
  companyName: string,
  subdomain: string,
  branding: {
    logo: string (url),
    primaryColor: string,
    secondaryColor: string
  },
  subscription: {
    plan: string,
    startDate: timestamp,
    expiryDate: timestamp,
    status: enum['active', 'suspended', 'cancelled']
  },
  settings: object,
  createdAt: timestamp,
  adminUserId: string
}
```

8. Non-Functional Requirements

8.1 Performance

- **Response Time:**
 - Page load: < 2 seconds
 - Search results: < 1 second
 - API calls: < 500ms (95th percentile)
- **Scalability:**
 - Support 5-10 concurrent users initially
 - Handle 200-500 customer records
 - Process 100-300 daily log entries
 - Scale to 1000+ customers without re-architecture

8.2 Security

- **Authentication:**
 - Firebase Auth with email/password
 - Session timeout after 24 hours
 - Secure password requirements
- **Data Protection:**
 - HTTPS/TLS for all communications
 - Encrypted data at rest (Firebase default)
 - Regular backups (daily)
 - GDPR compliance considerations
- **Access Control:**
 - Strict RBAC enforcement
 - Tenant data isolation
 - API rate limiting
 - Audit logs for sensitive operations

8.3 Availability

- **Uptime:** 99% availability target
- **Backup:** Daily automated backups, 30-day retention
- **Disaster Recovery:** Restore from backup within 4 hours

8.4 Usability

- **Design System:**
 - Dark theme with purple undertones ( #1a1a2e base,  #6C63FF purple accent)
 - Modern, clean interface
 - Consistent component library
 - Responsive design (desktop, tablet, mobile)
- **Accessibility:**
 - WCAG 2.1 Level AA compliance (goal)
 - Keyboard navigation support
 - Screen reader compatibility
 - High contrast mode option
- **User Experience:**
 - Intuitive navigation
 - < 3 clicks to common actions Contextual
 - help tooltips
 - Onboarding tutorial for new users

8.5 Compatibility

- -
 -
 -
 - Edge (last 2 versions)
- **Mobile:**
 - Android 8.0+ (Kotlin app)
 - iOS via web (responsive design)

Web Browsers:

Chrome (last 2 versions)

Firefox (last 2 versions)

Safari (last 2 versions)

9. Integration Specifications

9.1 Gmail API via n8n

- **Workflow:** Gmail → n8n → Python Backend → Firebase
- **Triggers:** New email received, email sent
- **Actions:** Fetch email, parse content, create log entry, auto-assign
- **Rate Limits:** Respect Gmail API quotas

9.2 Telegram Bot (BotFather)

- **Use Cases:**
 - Notifications (new complaint, mention, task due)
 - Quick queries ("Show my tasks")
 - Customer-facing bot (external interface)
- **Commands:** /status, /search, /create, /help

-

Integration: Webhook from Telegram to Python backend

9.3 Taiga API

- **Authentication:** OAuth 2.0 or API key
- **Operations:**
 - Create issue
 - Update issue status
 - Sync comments
 - Get issue details
- **Webhook:** Taiga → CRM for status updates
- **Mapping:** CRM complaint ↔ Taiga user story/issue

9.4 VoIP Integration (TBD - to be selected)

- **Options:** Twilio, Vonage, or existing PBX system
 - **Features:** Click-to-call, call logging, recording
 - **Protocol:** SIP/WebRTC
-

10. Development Phases

Phase 1: MVP (Core Features)

Timeline: 2 weeks (as per your requirement)

Sprint 1 (Week 1: Days 1-7)

Days 1-2: Foundation Setup

- Initialize project repositories (frontend, backend, mobile)
- Set up Firebase project (Firestore, Auth)
- Configure development environments
- Create base project structure
- Set up TailwindCSS with dark theme configuration
- Create design system components (buttons, inputs, cards)

Days 3-4: Authentication & Multi-Tenancy

- Implement Firebase Auth (email/password)
- Build login/signup UI
- Create tenant management system
- Implement RBAC middleware
- Build user invitation system
- Create organization settings page

Days 5-7: Customer Management

- Build customer CRUD API endpoints
- Create customer list UI
- Build customer detail view
- Implement basic search functionality
- Create customer form (add/edit)
- Add customer deletion (soft delete)

Sprint 2 (Week 2: Days 8-14)

Days 8-9: Logging System

-
- Build log entry API endpoints
- Create log entry form UI
- Implement file upload for attachments
- Build timeline view component
- Add log filtering and sorting
- Implement thread grouping for multiple histories

Days 10-11: Complaint Management

- Build complaint CRUD API endpoints
 - Create complaint form UI
 - Implement complaint status workflow
 - Build complaint list/board views
- Add SLA tracking logic
- Create complaint detail modal

Days 12-13: Email Integration

- Set up n8n workflows for Gmail
- Create email sync endpoints
- Build email list view UI
- Implement email threading
- Add email composition feature
- Set up automated email logging

Day 14: Testing & Polish

- Integration testing across all features
- Bug fixes
- Performance optimization
- UI/UX refinements
- Documentation updates
- Deployment preparation

Phase 1 Success Criteria:

- ✓ Users can authenticate and manage their accounts
- ✓ Multi-tenant architecture working with data isolation
- ✓ CRUD operations for customers functional
- ✓ Logging system operational with file attachments
- ✓ Complaint management system working end-to-end
- ✓ Email sync functioning with Gmail
- ✓ Basic search working across customers
- ✓ Responsive UI working on web and mobile browsers

Phase 2: Automation & Intelligence

Timeline: 6 weeks

Week 3-4: Email Automation & Search

- Implement automated email sorting (n8n workflows)
- Build email categorization rules engine
- Enhance search with Firestore full-text capabilities
- Add advanced filters and saved searches
- Create auto-assignment logic for emails
- Build email templates system

Week 5-6: Third-Party Integrations

-
- Integrate Taiga API
- Build issue sync mechanism
- Create Telegram bot (BotFather)
- Implement bot command handlers
- Add webhook receivers for external systems
- Build notification system

Week 7-8: Conversational Interface

- Implement NLP engine for internal chatbot
- Build intent recognition system
- Create conversation UI components
- Add context management
- Implement quick actions from chat
- Build analytics dashboard

Phase 2 Success Criteria:

- ✓ 80%+ email auto-categorization accuracy
 - ✓ Taiga issues bidirectionally synced
 - ✓ Telegram bot responding to commands
 - ✓ Chatbot handling 70%+ of queries correctly
 - ✓ Analytics dashboard showing key metrics
-

Phase 3: Enhanced Communication

Timeline: 6 weeks

Week 9-10: VoIP Integration

- Select and integrate VoIP provider (e.g., Twilio)
- Implement click-to-call functionality
- Build call logging system
- Add call recording capability
- Create call history view
- Implement WebRTC for browser calls

Week 11-12: Voice Commands

- Integrate speech-to-text API
- Build voice command recognition
- Implement command execution engine
- Add voice feedback system
- Create voice input UI
- Optimize for accuracy and speed

Week 13-14: Mobile App Development

- Build Android app UI (Kotlin)
- Implement API integration
- Add native features (camera, dialer)
- Build offline mode with sync
- Implement push notifications
- Add biometric authentication
- Testing and deployment to Play Store

-

Phase 3 Success Criteria:

- ✓ Calls can be initiated from web and mobile
 - ✓ Voice commands working with 85%+ accuracy
 - ✓ Mobile app feature parity with web
 - ✓ Offline mode functional
 - ✓ App published on Google Play Store
-

Phase 4: Optimization & Scaling

Timeline: Ongoing

Features:

- Performance optimization (query optimization, caching)
 - Advanced AI/ML features (sentiment analysis, predictions)
 - Multi-language support
 - iOS app development (if needed)
 - Additional integrations (WhatsApp, Slack)
 - Advanced reporting and BI
 - Custom workflow builder
 - White-label capabilities
-

11. AI Development Workflow

11.1 Day-by-Day Development Approach

Daily Structure:

1. **Morning:** Plan the day's features with AI co-pilot
2. **Development:** Implement feature incrementally
3. **Testing:** Test each component as you build

4. **Evening:** Review progress, document learnings

11.2 AI Co-Pilot

InstructionsFor

Backend Development:

"Create a FastAPI endpoint for [feature] that:

- Accepts [parameters]
- Validates [requirements]
- Performs [business logic]
- Returns [response format]
- Handles errors [scenarios]
- Includes Firebase Firestore queries with tenant filtering"

For Frontend Development:

"Create a React component for [feature] that:

- Uses TailwindCSS with our color scheme (#1a1a2e base, #6C63FF purple)
- Includes [UI elements]
- Handles [user interactions]
- Calls API endpoint [endpoint name]
- Shows loading and error states
- Is responsive for mobile"

For Mobile Development:

"Create a Kotlin fragment/activity for [feature] that:

- Follows Material Design guidelines
- Uses dark theme
- Implements [functionality]
- Integrates with [API endpoint]
- Handles offline scenarios"

11.3 Testing

StrategyUnit Tests:

- Test business logic functions
- Test API endpoint responses
- Test data validation

Integration Tests:

- Test API + Database interactions
- Test authentication flow
- Test multi-tenant data isolation

E2E Tests:

- Test critical user flows (login → create customer → add log)
- Test complaint workflow
- Test email sync

Manual Testing Checklist:

- ☐ All CRUD operations work
- ☐ Search returns accurate results
- ☐ Permissions enforced correctly
- ☐ Email sync working
- ☐ UI responsive on mobile
- ☐ Dark theme consistent
- ☐ No console errors

11.4 Code Organization

/crm-backend (Python/FastAPI)

```
|— /api
|   |— /routes
|   |   |— auth.py
|   |   |— customers.py
|   |   |— logs.py
|   |   |— complaints.py
|   |   |— search.py
|   |— /middleware
|   |   |— auth.py
|   |   |— tenant.py
|— /models
|   |— customer.py
|   |— log.py
|   |— complaint.py
|   |— user.py
|— /services
```

| |— firebase.py

- | |— email.py
- | |— taiga.py
- | |— telegram.py
- |— /utils
- | |— validation.py
- | |— helpers.py
- |— main.py
- |— requirements.txt

crm-frontend (React)

- |— /src
 - | |— /components
 - | | |— /common (Button, Input, Card, etc.)
 - | | |— /customers
 - | | |— /complaints
 - | | |— /logs
 - | |— /pages
 - | | |— Dashboard.jsx
 - | | |— Customers.jsx
 - | | |— CustomerDetail.jsx
 - | | |— Complaints.jsx
 - | |— /services
 - | | |— api.js
 - | |— /hooks
 - | | |— useAuth.js
 - | | |— useCustomers.js
 - | | |— useComplaints.js
 - | |— /utils
 - | | |— helpers.js
 - | |— App.jsx
 - | |— index.js
- |— tailwind.config.js
- |— package.json

crm-mobile (Kotlin)

- |— /app
- | |— /data


```

| | |— /models
| | |— /repositories
| | |— /api
| |— /domain
| |— /usecases
|— /presentation

| |— /ui
| |— /viewmodels
|— build.gradle

```

12. Success Metrics & KPIs

12.1 User Metrics

- **Daily Active Users (DAU):** Target 80% of invited users
- **User retention rate:** 90% week-over-week
- **Feature adoption rate:** 60% users use each feature within 2 weeks
- **Average session duration:** > 10 minutes

12.2 Performance Metrics

- **Page load time:** < 2 seconds (95th percentile)
- **API response time:** < 500ms (95th percentile)
- **Search latency:** < 1 second
- **Email sync delay:** < 5 minutes

12.3 Business Metrics

- **Customer records per user:** Average 15-30
- **Logs created per day:** 10-20 per active user
- **Complaint resolution time:** < 48 hours average
- **Email auto-categorization accuracy:** > 80%
-

Chatbot deflection rate: > 60%

12.4 Quality Metrics

- **Bug rate:** < 1 critical bug per 100 user sessions
 - **System uptime:** > 99%
 - **Data sync accuracy:** 100%
 - **User satisfaction (NPS):** > 40
-

13. Risks & Mitigations

13.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
Firebase costs exceed budget	Medium	High	Implement pagination, optimize queries, monitor usage dashboard daily
Gmail API quota limits hit	High	Medium	Implement exponential backoff, batch requests, queue system, consider workspace account
VoIP integration more complex than expected	Medium	High	Start with Twilio (well-documented), allocate extra time, have backup provider
Multi-tenancy data leakage	Low	Critical	Add tenant_id to ALL queries, write comprehensive tests, security audit before launch
Real-time sync performance issues	Medium	Medium	Implement debouncing, use Firebase onSnapshot wisely, add loading states
Mobile app offline mode complexity	High	Medium	Use local SQLite, implement sync queue, test extensively

13.2 Development Risks

Risk	Probability	Impact	Mitigation Strategy
2-week timeline too aggressive	High	High	Prioritize P0 features only, cut scope if needed, extend to 3 weeks if critical
AI co-pilot generates buggy code	Medium	Medium	Always review generated code, test incrementally, maintain good prompts
Integration APIs change unexpectedly	Low	High	Pin API versions, subscribe to change logs, have fallback plans
Scope creep during development	High	Medium	Stick to PRD, create "Future Enhancements" list, resist adding features

13.3 Operational Risks

Risk	Probability	Impact	Mitigation Strategy
Data loss	Low	Critical	Daily automated backups, 30-day retention, test restore process monthly
Service downtime	Medium	High	Set up health monitoring (UptimeRobot), configure alerts, have rollback plan
Security breach	Low	Critical	Regular security audits, encrypt sensitive data, implement rate limiting, log all access

Risk	Probability	Impact	Mitigation Strategy
Firebase service outage	Low	High	Monitor Firebase status page, have maintenance page ready, communicate with users

14. Security & Compliance

14.1 Authentication & Authorization

Firestore Auth Configuration:

- Email/password authentication
- Password requirements: Minimum 8 characters, 1 uppercase, 1 number, 1 special character
- Session timeout: 24 hours
- Failed login attempts: Lock account after 5 failed attempts within 15 minutes
- Password reset: Email-based with 1-hour expiration link

JWT Token Management:

- Access token expiration: 1 hour
- Refresh token expiration: 7 days
- Token rotation on refresh
- Store tokens securely (HttpOnly cookies for web)

Role-Based Access Control:

javascript

```

// Permission check middleware
const checkPermission = (requiredRole, requiredPermission) => {
  return async (req, res, next) => {
    const user = req.user;
    const tenant = req.tenant;

    // Check role hierarchy
    if (!hasRole(user, requiredRole)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }

    // Check specific permission
    if (requiredPermission && !hasPermission(user, requiredPermission)) {
      return res.status(403).json({ error: 'Permission denied' });
    }

    // Verify tenant access
    if (!canAccessTenant(user, tenant)) {
      return res.status(403).json({ error: 'Tenant access denied' });
    }

    next();
  };
};

```

14.2 Data Security

Encryption:

- All data in transit: TLS 1.3
- Data at rest: Firebase default encryption
- Sensitive fields (phone numbers, addresses): Additional application-level encryption
- API keys: Stored in environment variables, never in code

Data Access Logging:

javascript

```
// Audit log for sensitive operations
const auditLog = {
  userId: string,
  tenantId: string,
  action: string, // 'create', 'read', 'update', 'delete'
  entity: string, // 'customer', 'complaint', etc.
  entityId: string,
  changes: object, // before/after for updates
  ipAddress: string,
  userAgent: string,
  timestamp: timestamp
};
```

Firestore Security Rules:

javascript

rules_version

```

rules_version == '2';;
service cloudservice
cloud.firestorefirestore
{{
  match match //databasesdatabases/{ {databasedatabase} } //documents documents {{

```

// Helper functions//

Helper functions

```

functionfunction
isAuthenticatedisAuthenticate
d() {{      returnreturn
request request..authauth !=!=
nullnull;;
  }}

```

```

functionfunction
belongsToTenantbelongsToTenant((tenantIdt
enantId)) {{      returnreturn
isAuthenticatedisAuthenticated() &&&&
  request
request..authauth..token..tenantIdtenantId
=== tenantId tenantId;;    }}

```

```

functionfunction
hasRolehasRole((rolerole)) {{
returnreturn
isAuthenticatedisAuthenticated()
&&&&      request
request..authauth..token..rolerole
=== role role;;
  }}

```

// Customers collection//

```

Customers collection  match
match
//customerscustomers/{ {customerI
dcustomerId} } {{

```

```

allow read    allow read:: if belongsToTenant belongsToTenant((resource..data..tenantId));
allow create  allow create:: if
belongsToTenant belongsToTenant((request..resource..data..
tenantId)) &&&
hasRole hasRole(('admin'admin')) ||| hasRole hasRole(('manager'manager')) |||
hasRole hasRole(('sales_rep'sales_rep'));    allow update    allow update::
if
belongsToTenant belongsToTenant((resource..data..tenantId
d)) &&&
hasRole hasRole(('admin'admin')) |||
hasRole hasRole(('manager'manager'));
    allow    allow delete delete:: if
belongsToTenant belongsToTenant((resource..data..tenantId
) &&&
hasRole hasRole(('admin'admin'));
    }}

    // Logs collection//
Logs collection    match
match
//logs logs/{logId}
} {{
    allow read    allow read:: if belongsToTenant belongsToTenant((resource..data..tenantId));
    allow create  allow create:: if
belongsToTenant belongsToTenant((request..resource..data..tenantId
tenantId)) &&&
isAuthenticated isAuthenticated(());
    allow update  allow update, delete delete:: if
belongsToTenant belongsToTenant((resource..data..tenantId)) &&&

((resource..data..createdBy == request
request..auth..uid
hasRole hasRole(('admin'admin')));
    }}

    // Complaints collection// Complaints collection
match    match //complaints complaints/{complaintId} {{
    allow read    allow read:: if belongsToTenant belongsToTenant((resource..data..tenantId));

```

```

allow create: if isAuthenticated();
allow update: if belongsToTenant(resource.data.tenantId) &&
    (hasRole('admin') ||
    hasRole('manager') ||
    resource.data.assignedTo === request.auth.uid);
allow delete: if belongsToTenant(resource.data.tenantId) &&
    hasRole('admin');
}

// Tenants collection
match /tenants/{tenantId} {
  allow read: if belongsToTenant(tenantId);
  allow write: if hasRole('super_admin') ||
    (belongsToTenant(tenantId) && hasRole('tenant_admin'));
}

// Users collection
match /users/{userId} {
  allow read: if isAuthenticated() &&
    (userId === request.auth.uid || hasRole('admin'));
  allow write: if hasRole('admin') || hasRole('tenant_admin');
}
}
}

```

14.3 API Security

Rate Limiting:

python

```
# Implement rate limiting per user
@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next):
    user_id = request.state.user_id

    # Check rate limit (100 requests per minute)
    key = f'rate_limit:{user_id}'
    count = await redis.incr(key)

    if count == 1:
        await redis.expire(key, 60) # 60 seconds

    if count > 100:
        return JsonResponse(
            status_code=429,
            content={"error": "Rate limit exceeded"}
        )

    response = await call_next(request)
    return response
```

Input Validation:

python

```
from pydantic import BaseModel, EmailStr, validator

class CustomerCreate(BaseModel):
    companyName: str
    contactPerson: ContactPerson

    @validator('companyName')
    def validate_company_name(cls, v):
        if len(v) < 2 or len(v) > 100:
            raise ValueError('Company name must be 2-100 characters')
        return v.strip()

class Config:
    # Prevent SQL injection, XSS
    anystr_strip_whitespace = True
```

CORS Configuration:

python

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://yourcrm.com",
        "https://*.yourcrm.com", # Allow subdomains
        "http://localhost:3000" # Development only
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)
```

14.4 GDPR Compliance

Data Export:

python

```
@app.get("/users/me/export")
async def export_user_data(current_user: User):
    """Export all user data (GDPR right to data portability)"""

    # Gather all user data
    data = {
        "profile": current_user.dict(),
        "customers": await get_user_customers(current_user.id),
        "logs": await get_user_logs(current_user.id),
        "complaints": await get_user_complaints(current_user.id),
        "emails": await get_user_emails(current_user.id)
    }

    # Return as downloadable JSON
    return JSONResponse(content=data)
```

Data Deletion:

python

```
@app.delete("/users/me")
async def delete_user_account(current_user: User):
    """Delete user account and all associated data (GDPR right to erasure)"""

    # Soft delete user
    await mark_user_deleted(current_user.id)

    # Anonymize all logs/activities
    await anonymize_user_data(current_user.id)

    # Schedule hard delete after 30 days
    await schedule_hard_delete(current_user.id, days=30)

    return {"message": "Account scheduled for deletion"}
```


Privacy Policy Requirements:

- What data is collected
- How data is used
- How long data is retained
- Third-party services used (Firebase, Gmail, Taiga, Twilio)
- User rights (access, export, delete)
- Contact information for privacy concerns

14.5 Security Checklist

Before Launch:

- ☐ All API endpoints require authentication
- ☐ Tenant isolation verified (cross-tenant queries impossible)
- ☐ HTTPS enforced on all endpoints
- ☐ Firestore security rules tested and deployed
- ☐ Rate limiting implemented
- ☐ Input validation on all endpoints
- ☐ XSS prevention (sanitize HTML inputs)
- ☐ SQL injection prevention (use parameterized queries)
- ☐ Secrets stored in environment variables
- ☐ Error messages don't leak sensitive info
- ☐ Audit logging for sensitive operations
- ☐ Password reset flow secure
- ☐ Session management secure
- ☐ CORS properly configured
- ☐ Dependencies updated (no known vulnerabilities)

Ongoing:

- ☐ Regular security audits (quarterly)
 - ☐ Penetration testing (before major releases)
 - ☐ Monitor Firebase Auth logs for suspicious activity
 - ☐ Review audit logs weekly
 - ☐ Update dependencies monthly
 - ☐ Backup restoration test monthly
-

15. Performance Optimization

15.1 Database Optimization

Firestore Indexing:

```
javascript

// Create composite indexes for common queries
// customers collection
{
  "collectionGroup": "customers",
  "fields": [
    { "fieldPath": "tenantId", "order": "ASCENDING" },
    { "fieldPath": "status", "order": "ASCENDING" },
    { "fieldPath": "createdAt", "order": "DESCENDING" }
  ]
}

// complaints collection
{
  "collectionGroup": "complaints",
  "fields": [
    { "fieldPath": "tenantId", "order": "ASCENDING" },
    { "fieldPath": "status", "order": "ASCENDING" },
    { "fieldPath": "severity", "order": "ASCENDING" }
  ]
}
```

Query Optimization:

python

Bad: Loading all customers then filtering

```
all_customers = db.collection('customers').get()
active_customers = [c for c in all_customers if c.status == 'active']
```

Good: Filter at database level

```
active_customers = (
    db.collection('customers')
    .where('tenantId', '==', tenant_id)
    .where('status', '==', 'active')
    .limit(20)
    .get()
)
```

Pagination:

python

Implement cursor-based pagination

```
@app.get("/customers")
async def get_customers(
    tenant_id: str,
    limit: int = 20,
    start_after: str = None # Last document ID from previous page
):
    query = (
        db.collection('customers')
        .where('tenantId', '==', tenant_id)
        .order_by('createdAt', direction='DESCENDING')
        .limit(limit)
    )

    if start_after:
        start_doc = db.collection('customers').document(start_after).get()
        query = query.start_after(start_doc)

    customers = query.get()
    return customers
```

15.2 Frontend Optimization

Code Splitting:

```
javascript

// Lazy load routes
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./pages/Dashboard'));
const Customers = lazy(() => import('./pages/Customers'));
const Complaints = lazy(() => import('./pages/Complaints'));

function App() {
  return (
    <Suspense fallback={<Loading />}>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/customers" element={<Customers />} />
        <Route path="/complaints" element={<Complaints />} />
      </Routes>
    </Suspense>
  );
}
```

Caching:

javascript

```
// Use React Query for API caching
import { useQuery } from '@tanstack/react-query';

function CustomerList() {
  const { data, isLoading } = useQuery(
    ['customers', tenantId],
    () => fetchCustomers(tenantId),
    {
      staleTime: 5 * 60 * 1000, // 5 minutes
      cacheTime: 10 * 60 * 1000 // 10 minutes
    }
  );
}
```

Image Optimization:

- Use WebP format for images
- Lazy load images below fold
- Compress uploaded images (max 1MB)
- Use CDN for static assets

15.3 Backend Optimization

Caching with Redis:

python

```
import redis
```

```
import json
```

```
redis_client = redis.Redis(host='localhost', port=6379)
```

```
async def get_customer(customer_id: str):
```

```
    # Try cache first
```

```
    cache_key = f'customer:{customer_id}'
```

```
    cached = redis_client.get(cache_key)
```

```
    if cached:
```

```
        return json.loads(cached)
```

```
    # Fetch from database
```

```
    customer = await db.collection('customers').document(customer_id).get()
```

```
    # Cache for 5 minutes
```

```
    redis_client.setex(cache_key, 300, json.dumps(customer))
```

```
    return customer
```

Background Jobs:

python

```
# Use Celery for long-running tasks
from celery import Celery

celery_app = Celery('crm', broker='redis://localhost:6379')

@celery_app.task
def process_email_batch(email_ids: list):
    """Process emails in background"""
    for email_id in email_ids:
        email = fetch_email(email_id)
        categorize_email(email)
        link_to_customer(email)
        notify_assigned_user(email)
```

Database Connection Pooling:

```
python

# Reuse Firebase connections
from google.cloud import firestore
from functools import lru_cache

@lru_cache()
def get_db():
    """Singleton database connection"""
    return firestore.Client()

db = get_db()
```

16. Monitoring & Analytics

16.1 Application Monitoring

Health Check Endpoint:

python

```
@app.get("/health")
async def health_check():
    """Check system health"""
    checks = {
        "api": "ok",
        "database": await check_database(),
        "firebase_auth": await check_firebase_auth(),
        "redis": await check_redis(),
        "email_service": await check_email_service()
    }

    status = "healthy" if all(v == "ok" for v in checks.values()) else "degraded"

    return {
        "status": status,
        "checks": checks,
        "timestamp": datetime.utcnow().isoformat()
    }
```

Error Tracking:

python

```
import sentry_sdk

sentry_sdk.init(
    dsn="your-sentry-dsn",
    traces_sample_rate=0.1,
    environment="production"
)

@app.exception_handler(Exception)
async def handle_exception(request: Request, exc: Exception):
    # Log to Sentry
    sentry_sdk.capture_exception(exc)

    # Return user-friendly error
    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error"}
    )
```

Performance Monitoring:

python

```
from prometheus_client import Counter, Histogram, generate_latest

# Metrics
request_count = Counter('http_requests_total', 'Total HTTP requests')
request_duration = Histogram('http_request_duration_seconds', 'HTTP request duration')

@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    start_time = time.time()
    request_count.inc()

    response = await call_next(request)

    duration = time.time() - start_time
    request_duration.observe(duration)

    return response

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

16.2 User Analytics

Track Key Events:

javascript

```
// Frontend analytics
import Analytics from 'analytics';

const analytics = Analytics({
  app: 'crm-app',
  plugins: [
    // Google Analytics, Mixpanel, etc.
  ]
});

// Track events
analytics.track('customer_created', {
  customerId: customer.id,
  status: customer.status
});

analytics.track('complaint_resolved', {
  complaintId: complaint.id,
  resolutionTime: resolutionTime
});

analytics.track('email_sent', {
  recipientCount: recipients.length,
  hasAttachment: attachments.length > 0
});
```

Analytics Dashboard Queries:

python

Daily active users

```
SELECT COUNT(DISTINCT user_id) as dau
FROM events
WHERE event_date = CURRENT_DATE;
```

Feature adoption

```
SELECT
    feature_name,
    COUNT(DISTINCT user_id) as unique_users,
    COUNT(*) as total_uses
FROM events
WHERE event_date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
GROUP BY feature_name;
```

Complaint resolution metrics

```
SELECT
    AVG(TIMESTAMP_DIFF(resolved_at, created_at, HOUR)) as avg_resolution_hours,
    severity,
    COUNT(*) as count
FROM complaints
WHERE status = 'resolved'
GROUP BY severity;
```

16.3 Alerting

Set Up Alerts:

```
# Alert configuration (e.g., PagerDuty, Slack)
```

```
alerts:
```

- **name:** high_error_rate
condition: error_rate > 5%
window: 5m
severity: critical
notify: ["on-call-engineer", "slack-ops"]
- **name:** slow_api_response
condition: p95_latency > 2s
window: 10m
severity: warning
notify: ["slack-ops"]
- **name:** database_connection_errors
condition: db_connection_errors > 10
window: 5m
severity: critical
notify: ["on-call-engineer", "slack-ops"]
- **name:** low_disk_space
condition: disk_usage > 80%
window: 1h
severity: warning
notify: ["slack-ops"]

17. Deployment Strategy

17.1 Environment Setup

Environments:

1. **Development:** Local machine, Firebase emulators
2. **Staging:** Cloud deployment, test data
3. **Production:** Cloud deployment, real data

Environment Variables:

```
bash
```

```
# .env.production
```

```
FIREBASE_API_KEY=your_api_key
```

```
FIREBASE_AUTH_DOMAIN=your_project.firebaseio.com
```

```
FIREBASE_PROJECT_ID=your_project_id
```

```
DATABASE_URL=https://your_project.firebaseio.com
```

```
GOOGLE_CLIENT_ID=your_gmail_client_id
```

```
GOOGLE_CLIENT_SECRET=your_gmail_client_secret
```

```
TAIGA_API_URL=https://api.taiga.io/api/v1
```

```
TAIGA_AUTH_TOKEN=your_taiga_token
```

```
TELEGRAM_BOT_TOKEN=your_telegram_bot_token
```

```
VOIP_API_KEY=your_voip_api_key
```

```
VOIP_SECRET=your_voip_secret
```

```
SENTRY_DSN=your_sentry_dsn
```

```
REDIS_URL=redis://localhost:6379
```

17.2 CI/CD Pipeline

GitHub Actions Workflow:

yaml

name: Deploy CRM Deploy CRM

```

onon::
pushpus
h::

branchesbranches::
[[mainmain]]
pull_requestpull_reque
st::
branchesbranches::
[[mainmain]]

jobsjobs:
:
testtest::
    runs-onruns-on:: ubuntu
ubuntu--latestlatest
stepssteps::
    -- usesuses:: actions/checkout@v2 actions/checkout@v2

    -- name: Set up Python Set up Python
    usesuses:: actions/setup
actions/setup--python@v2python@v2
withwith::
    python-versionpython-version:: '3.9'3.9

    -- name: Install
dependencies Install dependencies
runrun:: ||
    pip install -r requirements.txt    pip install -r requirements.txt

    -- name: Run tests Run tests
    runrun:: ||
    pytest tests/    pytest tests/

    -- name: Run linter Run linter

runrun:: ||

```

flake8 .

flake8 .

deploy-

backenddeploy-

backend::

needsneeds:: test test

ifif:: github.ref ==

'refs/heads/main' github.ref ==

'refs/heads/main' runs-onruns-

on:: ubuntu ubuntu--latestlatest

stepssteps::

-- usesuses:: actions/checkout@v2 actions/checkout@v2

-- name:: Deploy to Cloud Run Deploy to Cloud Run

usesuses:: google google--githubgithub--

actions/deployactions/deploy--cloudrun@v0cloudrun@v0

withwith::

serviceservice:: crm crm--apiapi

imageimage:: gcr.io/\$ gcr.io/\${{ secrets.GCP_PROJECT secrets.GCP_PROJECT }}/crm/crm--
apiapi::\${{ github.sha github.sha }}

`region: us-central1`

deploy-frontend:

`needs: test`

`if: github.ref == 'refs/heads/main'`

`runs-on: ubuntu-latest`

steps:

- `uses: actions/checkout@v2`

- `name: Build React app`

`run: |`

`npm install`

`npm run build`

- `name: Deploy to Firebase Hosting`

`uses: FirebaseExtended/action-hosting-deploy@v0`

`with:`

`repoToken: '${{ secrets.GITHUB_TOKEN }}'`

`firebaseServiceAccount: '${{ secrets.FIREBASE_SERVICE_ACCOUNT }}'`

`projectId: your-project-id`

17.3 Database Migration Strategy

Firestore Schema Versioning:

javascript

```

// Version all documents
{
  _version: 1,
  _createdAt: timestamp,
  _updatedAt: timestamp,
  // ... rest of data
}

// Migration script
async function migrateCustomersV1toV2() {
  const customers = await db.collection('customers')
    .where('_version', '==', 1)
    .get();

  const batch = db.batch();

  customers.forEach(doc => {
    const ref = db.collection('customers').doc(doc.id);
    batch.update(ref, {
      _version: 2,
      // Add new fields or transform existing ones
      tags: doc.data().categories || [] // Rename field
    });
  });

  await batch.commit();
  console.log(`Migrated ${customers.size} customers to v2`);
}

```

17.4 Rollback Plan

Quick Rollback Process:

1. Identify issue in production
2. Revert to previous Git commit
3. Trigger CI/CD deployment of stable version
4. Monitor health check endpoints

5. Communicate status to users

Database Rollback:

- Maintain database backups before each deployment
 - Test restore process regularly
 - Document rollback steps clearly
-

18. Documentation Requirements

18.1 Developer Documentation

README.md Structure:

markdown

CRM System

Quick Start

1. Clone repository
2. Install dependencies
3. Configure environment variables
4. Run development server

Architecture

- Tech stack overview
- System architecture diagram
- Data flow diagrams

API Documentation

- See /docs/api.md
- Swagger/OpenAPI spec at /api/docs

Development

- Code structure
- Coding standards
- Testing guidelines
- Contributing guidelines

Deployment

- Environment setup
- CI/CD pipeline
- Deployment process

Troubleshooting

- Common issues
- Debugging tips
- FAQ

18.2 User Documentation

User Guide Sections:

1. Getting Started

- Account creation
- First login
- Interface overview

2. Managing Customers

- Adding customers
- Viewing customer history
- Searching customers

3. Logging Activities

- Creating logs
- Attaching files
- Organizing with threads

4. Handling Complaints

- Submitting complaints
- Tracking status
- Resolution process

5. Email Integration

- Connecting Gmail
- Sending emails
- Email automation

6. Using Conversational Interface

- Chat commands
- Voice commands
- Tips and tricks

7. Admin Guide

- User management
- Organization settings
- Integrations setup
- Billing management

18.3 API Documentation

Auto-generated with FastAPI:

```
python
```

```
from fastapi import FastAPI
from fastapi.openapi.utils import get_openapi

app = FastAPI(
    title="CRM API",
    description="Modern CRM System API Documentation",
    version="1.0.0",
    docs_url="/api/docs",
    redoc_url="/api/redoc"
)

# Custom OpenAPI schema
def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema

    openapi_schema = get_openapi(
        title="CRM API",
        version="1.0.0",
        description="Complete API documentation for CRM system",
        routes=app.routes,
    )

    app.openapi_schema = openapi_schema
    return app.openapi_schema

app.openapi = custom_openapi
```

19. Future Enhancements (Post-MVP)

19.1 Phase 4+ Features

Communication Enhancements:

- WhatsApp Business API integration
- SMS notifications
- Video call integration (Zoom, Google Meet)
- Social media monitoring (Twitter, LinkedIn)

Workflow Automation:

- Visual workflow builder (drag-and-drop)
- Custom automation rules
- Zapier/Make.com integration
- Scheduled tasks and reminders

Advanced Analytics:

- Predictive analytics (churn prediction, upsell opportunities)
- Custom report builder
- Data export to BI tools (Tableau, Power BI)
- Sentiment analysis on customer communications

AI/ML Features:

- Smart email responses (AI-generated drafts)
- Lead scoring
- Automatic task prioritization
- Customer segmentation
- Anomaly detection

Mobile Enhancements: iOS

- app (Swift/SwiftUI)
- Offline-first architecture
-

Mobile-specific features (GPS check-in, business card scanner) **Collaboration:**

- Internal chat/messaging
- @mentions and notifications
- Shared notes and comments
- Team inbox for emails

Sales & Marketing:

- Pipeline management
- Deal tracking
- Quote and proposal generation
- Invoice creation
- Payment processing integration
- Email campaigns
- Landing page builder
- Lead capture forms

Advanced Integrations:

- Calendar sync (Google Calendar, Outlook)
- File storage (Google Drive, Dropbox)
- Accounting software (QuickBooks, Xero)
- E-commerce platforms (Shopify, WooCommerce)
- Contract management (DocuSign)

Enterprise Features:

- Advanced security (SSO, SAML)
 - Audit trail and compliance reporting
 - Custom branding (white-label)
 - API for custom integrations
 - Webhooks for external systems
 - Multi-language support
 - Advanced user permissions (custom roles)
-

20. Appendix

20.1 Glossary

Term	Definition
CRM	Customer Relationship Management - system for managing interactions with customers
Tenant	An organization using the CRM (multi-tenant architecture)

Term	Definition
RBAC	Role-Based Access Control - permission system based on user roles
SLA	Service Level Agreement - commitment to response/resolution time
Thread	A conversation or interaction stream with a customer
Log	A record of an interaction, activity, or note
Complaint	A customer issue that needs resolution
VoIP	Voice over IP - internet-based phone calls
JWT	JSON Web Token - authentication token format
API	Application Programming Interface
NLP	Natural Language Processing
Firestore	Google's NoSQL cloud database
n8n	Open-source workflow automation tool
Webhook	HTTP callback for real-time updates

20.2 References

Documentation:

- Firebase: <https://firebase.google.com/docs>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://react.dev/>
- TailwindCSS: <https://tailwindcss.com/>
- Kotlin: <https://kotlinlang.org/docs/>
- Gmail API: <https://developers.google.com/gmail/api>
- Taiga API: <https://docs.taiga.io/api.html>
- n8n: <https://docs.n8n.io/>
- Telegram Bot API: <https://core.telegram.org/bots/api>

Third-Party Services:

- Twilio (VoIP): <https://www.twilio.com/docs>
- Sentry (Error Tracking): <https://docs.sentry.io/>
- UptimeRobot (Monitoring): <https://uptimerobot.com/>
- Vercel/Netlify (Hosting): Respective documentation

20.3 Assumptions

1. Technical:

- Stable internet connectivity for users
- Modern browsers (last 2 versions)
- Firebase free tier sufficient for MVP
- Gmail API access obtainable
- Taiga API publicly accessible VoIP
- provider selected during Phase 3

2. Business:

- Target audience has basic computer literacy
- English language only in MVP
- SMB market focus initially
- Users willing to connect Gmail account Subscription-based
- revenue model (future)

3. Development:

- AI co-pilot (ChatGPT, Cursor, etc.) available
- You have intermediate programming knowledge
- Development machine meets requirements
- Access to necessary API keys
- Time to dedicate 6-8 hours daily for 2 weeks

4. Operational:

- You'll handle initial support
 - Gradual user onboarding (not thousands at once)
 - Willingness to iterate based on feedback
 - Accept some technical debt in MVP
-

21. Conclusion

This PRD provides a comprehensive blueprint for building your modern CRM system. With clear specifications, wireframes, API documentation, and a detailed development plan, you're equipped to build a production-ready application with AI assistance.

Key Success Factors:

1. **Focus on MVP:** Resist scope creep, deliver Phase 1 in 2 weeks
2. **Test Continuously:** Test each feature as you build
3. **Prioritize Security:** Implement tenant isolation and RBAC correctly from day 1
4. **Monitor Everything:** Set up logging, monitoring, and analytics early
5. **Document as You Go:** Keep documentation updated
6. **Iterate Based on Feedback:** Be ready to adapt based on real usage

Next Steps:

1. ☒ Review this PRD thoroughly
2. ☐ Set up development environment
3. ☐ Create Firebase project
4. ☐ Initialize Git repositories
5. ☐ Configure design system and components
6. ☐ Start Day 1 of Phase 1 development

Prepared By :

Syed Tahmid Manzoor

North South University

27 October 2025

END OF DOCUMENT